TaskTrack: A Python Console-Based System for Managing Academic Tasks

---

Introduction to Computing Project
Presented to the
Faculty of the College of Information Technology
Cagayan de Oro College
Cagayan de Oro City

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Science in Information Technology

NAME OF RESEARCHER

CARITAN, ROLLAN M.
NOVERAS, WRYL ACE S.
CARREON, JOHN DAVE L.
GABITO, MICAH DIANE P.
DANGGOY, ASHLEY JANE L.

OCTOBER 2026

1.1. Introduction

Digital task management uses computer-based technology to record, organize, and monitor academic tasks. In TaskTrack, this process is implemented through a Python console-based application where students enter the task title, deadline, time, category, and priority. The system stores each task in a Python list that assigns a Pending status, and processes the recorded information to display tasks, identify whether deadlines are Completed, Overdue, Due Today, or Upcoming, and calculate the total tasks, completed tasks, and completion rate. This process provides students with a simple way to organize and monitor academic responsibilities through a console interface without requiring a web-based or graphical system (Khaldi & Erradi, 2020; Santos et al., 2022; Shah et al., 2026).

Students commonly manage academic tasks with different deadlines, times, categories, priorities, and completion statuses. When these tasks are recorded manually or are not organized in one system, students may have difficulty identifying upcoming or overdue tasks and monitoring their overall progress. In the proposed setting, three specific problems are identified: difficulty recording task information consistently, difficulty viewing tasks and determining their deadline status, and difficulty monitoring overall task progress and removing tasks that are no longer needed.

To address these problems, the proponents developed Task-Track Campus: A Proposed Console-Based Task Management System for Recording and Managing Student Academic Tasks. The proposed system is a Python console-based application designed to allow students to record their tasks, view and summarize their task information, and delete completed or unnecessary records. These functions are intended to provide a simple and organized approach to managing academic task records.

1.2. Statement of the Problem

General Problem

Students may experience difficulty organizing and monitoring their academic tasks when requirements are recorded manually or are not consistently organized. This can make it difficult to keep track of deadlines, task details, and the current status of their responsibilities.

Specific Problems

The proposed system aims to address the following problems:

1. Difficulty recording academic tasks consistently.
2. Difficulty viewing and monitoring recorded task information.
3. Difficulty removing tasks that are no longer needed**.**

1.3. Objectives of the Study

General Objective

To develop Task-Track Campus, a console-based task management system that helps students organize, monitor, and manage their academic and school-related responsibilities.

Specific Objectives

1. To develop a feature that allows users to record academic task information.
2. To provide a feature for viewing recorded tasks and their details.
3. To provide a task summary to help users monitor their recorded tasks.
4. To provide a feature for deleting tasks that are no longer needed.

1.4. Scope

This project covers the TaskTrack: A Python Console-Based System for Managing Academic Tasks, intended for students who want to manage their academic and school-related tasks. The system operates on a personal computer and uses in-memory storage to maintain task records while the program is running.

The project includes the following functions:

- Add Task – The user can enter the task title, description, deadline, category, and priority. The system records the information for viewing and management.
- View Tasks – The user can display recorded tasks and review their details, including the deadline, category, priority, and status.
- Task Summary – The system provides a summary of the recorded tasks to help the user monitor their task information
- Update task -The update task function allows users to edit task details, such as the title, priority, due date, and status.
- Delete Task – The user can select and remove a task that is no longer needed from the task list.
- Mark as Done – The system checks selected inputs, such as required task information and task numbers, to reduce errors during operation.
- Exit task - The exit task allows the system to close gracefully without leaving or making a mess.

The project focuses only on basic academic task management and does not cover online task synchronization, cloud storage, automatic notifications, user accounts, or web-based and graphical interfaces.

1.5. Limitations

Task-Track Campus has the following limitations:

- The system operates only through a console-based interface and does not provide a graphical or web-based interface.
- Task records are stored temporarily in memory and may not be retained after the program is closed.
- The system does not provide automatic reminders or notifications for upcoming deadlines.

- The system is intended for basic academic task management and does not include advanced project-management features.
- The system does not require an online connection and does not include cloud-based data storage.
- The system is designed primarily for individual users and does not include multi-user or account management.

1.6. System Requirements

Hardware Requirements

The system requires basic computer hardware capable of running Python:

- Processor: Any processor can access our system
- RAM: At least 6 GB
- Storage: At least 3 GB of available space
- Display: Standard monitor / laptop display
- Keyboard: Required for entering task information
- Mouse: Optional for navigating the development environment

Software Requirements

The following software is required to run and develop the system:

- Operating System: Windows, Mac Os, Linux
- Programming Language: Python
- Code Editor/IDE: Visual Studio Code, IDLE, or another Python-compatible editor

The system uses Python's built-in programming features, including lists, dictionaries, functions, loops, conditional statements, and (match/case).

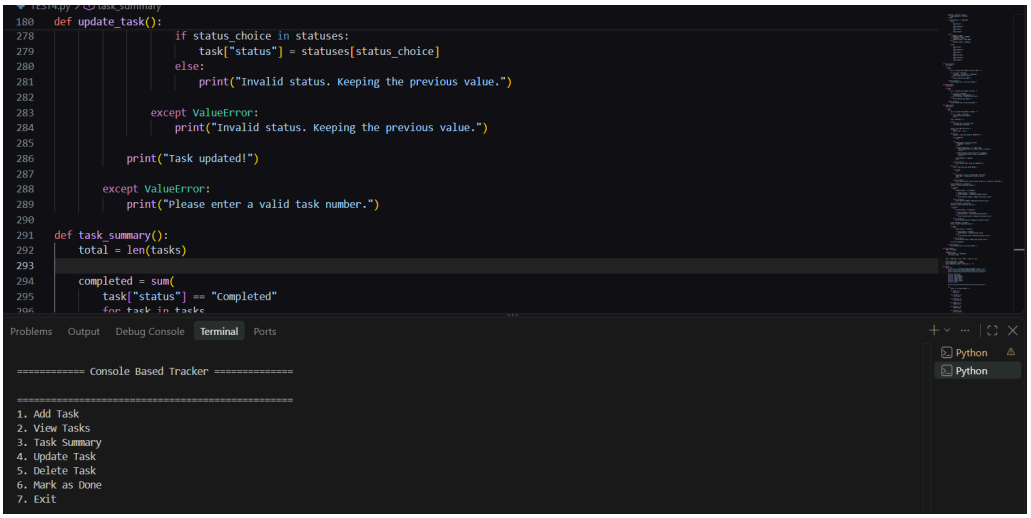
Functional Requirements

The system shall allow the user to:

- Add a task by entering the task title, description, deadline, category, and priority, after which the system will save the task in the task list.
- View tasks by selecting the View Tasks option, after which the system will display the recorded tasks and their details.
- View a task summary to display an overview of the recorded academic tasks.
- Update a task by selecting a recorded task and modifying its information to keep task details accurate and updated.
- Delete a task by selecting a recorded task, after which the system will remove it from the task list.
- Mark a task as completed to indicate which academic tasks have already been finished.
- Validate task inputs by displaying an appropriate message when required information is missing or an invalid option is entered.
- Exit the application by selecting the designated Exit option, after which the system will terminate the program.

1.7 Code Implementation of the TaskTrack Console-Based Task Management System

The following code features demonstrate selected functions implemented in the Task Management System. The code defines task categories, priorities, and statuses, which are used to organize and manage students' tasks efficiently.



```
def update_task():
                    if status_choice in statuses:
                        task["status"] = statuses[status_choice]
                    else:
                        print("Invalid status. Keeping the previous value.")

                except ValueError:
                    print("Invalid status. Keeping the previous value.")

            print("Task updated!")

        except ValueError:
            print("Please enter a valid task number.")

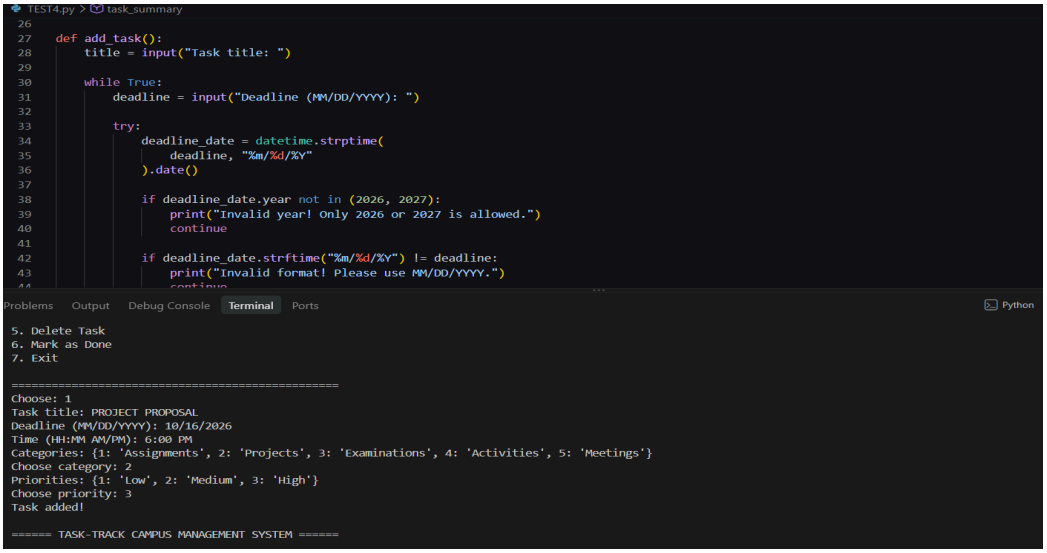
def task_summary():
    total = len(tasks)

    completed = sum(
        task["status"] == "Completed"
```

```
========== Console Based Tracker ============

==============================================
1. Add Task
2. View Tasks
3. Task Summary
4. Update Task
5. Delete Task
6. Mark as Done
7. Exit
```

Snippet 1 – Adding Task



```
def add_task():
    title = input("Task title: ")

    while True:
        deadline = input("Deadline (MM/DD/YYYY): ")

        try:
            deadline_date = datetime.strptime(
                deadline, "%m/%d/%Y"
            ).date()

            if deadline_date.year not in (2026, 2027):
                print("Invalid year! Only 2026 or 2027 is allowed.")
                continue

            if deadline_date.strftime("%m/%d/%Y") != deadline:
                print("Invalid format! Please use MM/DD/YYYY.")
```

```
5. Delete Task
6. Mark as Done
7. Exit

==============================================
Choose: 1
Task title: PROJECT PROPOSAL
Deadline (MM/DD/YYYY): 10/16/2026
Time (HH:MM AM/PM): 6:00 PM
Categories: {1: 'Assignments', 2: 'Projects', 3: 'Examinations', 4: 'Activities', 5: 'Meetings'}
Choose category: 2
Priorities: {1: 'Low', 2: 'Medium', 3: 'High'}
Choose priority: 3
Task added!

====== TASK-TRACK CAMPUS MANAGEMENT SYSTEM ======
```

Related Objective: To develop a feature that allows users to record academic tasks by entering the task title, deadline, time, category, and priority, with each task automatically assigned a "Pending" status.

Purpose: The purpose of the Add Task feature is to provide users with a simple way to record and organize their academic tasks and important task details in the system..

```
def add_task():
    title = input("Task title: ")
    while True:
        deadline = input("Deadline (MM/DD/YYYY): ")
        try:
            deadline_date = datetime.strptime(
                deadline, "%m/%d/%Y"
            ).date()
            if deadline_date.year not in (2026, 2027):
                print("Invalid year! Only 2026 or 2027 is allowed.")
                continue
            if deadline_date.strftime("%m/%d/%Y") != deadline:
                print("Invalid format! Please use MM/DD/YYYY.")
                continue
            break
        except ValueError:
            print(
                "Invalid date! Please use a valid MM/DD/YYYY "
                "with a valid month (1-12), day, and year (2026 or 2027)."
            )
    while True:
        time = input("Time (HH:MM AM/PM): ")
        try:
            valid_time = datetime.strptime(time, "%I:%M %p")
```



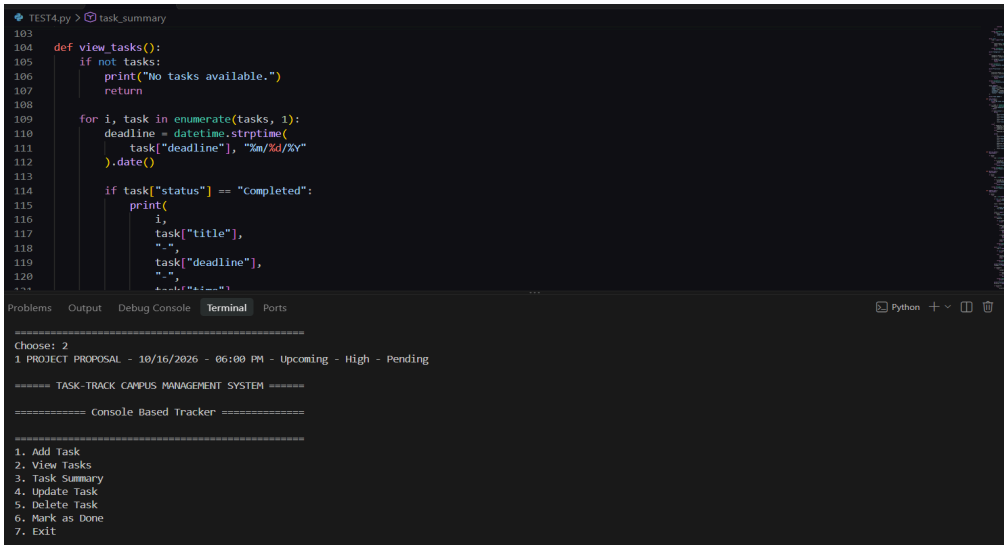
```
            time = valid_time.strftime("%I:%M %p")
            break
        except ValueError:
            print("Invalid time! Please use the 12-hour format (e.g., 09:30 AM or 02:45 PM).")
    print("Categories:", categories)
    try:
        category_choice = int(input("Choose category: "))
        category = categories.get(category_choice)
        if category is None:
            print("Invalid category!")
            return
    except ValueError:
        print("Invalid category!")
        return
    print("Priorities:", priorities)
    try:
        priority_choice = int(input("Choose priority: "))
        priority = priorities.get(priority_choice)
        if priority is None:
            print("Invalid priority!")
            return
    except ValueError:
        print("Invalid priority!")
        return
    tasks.append({
        "title": title,
        "deadline": deadline,
        "time": time,
        "category": category,
        "priority": priority,
        "status": "Pending"
```

```
})

print("Task added!")
```

The add_task() function is used to create and save a new task in the task management system. It first asks the user to enter the task title, deadline, and time. It then displays the available categories and priorities, allowing the user to select one by entering its number. The selected information is stored in a dictionary, along with the default status "Pending". Finally, the dictionary is added to the tasks list using append(), and the program displays "Task added!" to confirm that the task was successfully saved.

Snippet 2 – View Task



Related Objective: To develop a feature that allows users to view recorded academic tasks and identify whether each task is completed, overdue, due today, or upcoming.

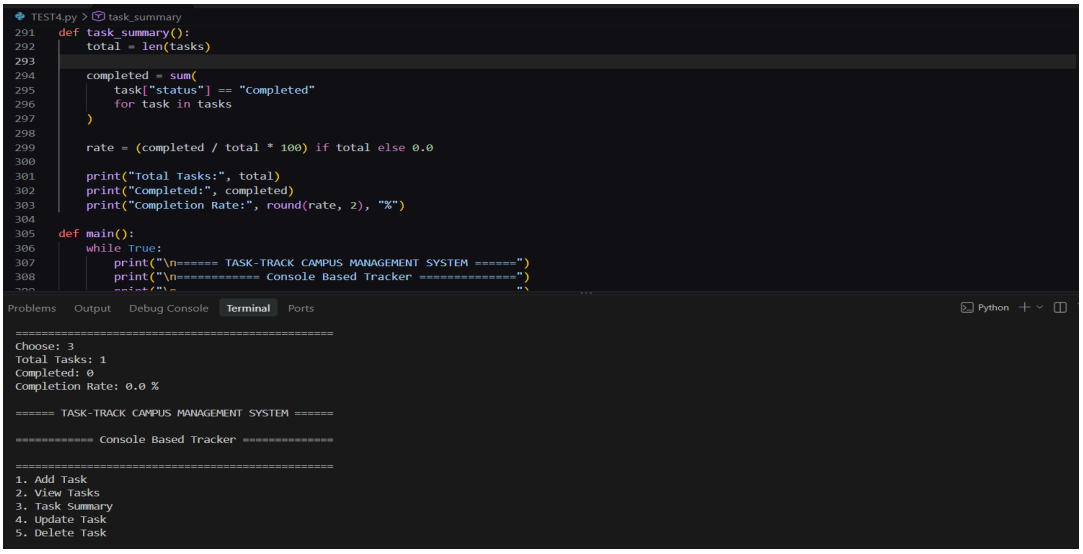
Purpose: The purpose of the View Tasks feature is to help users monitor their recorded tasks, deadlines, time, priority, and current status in an organized display.

```
def view_tasks():
    if not tasks:
        print("No tasks available.")
        return
    for i, task in enumerate(tasks, 1):
        deadline = datetime.strptime(
            task["deadline"], "%m/%d/%Y"
        ).date()
        if task["status"] == "Completed":
            print(
                i,
                task["title"],
                "-",
                task["deadline"],
                "-",
                task["time"],
                "-",
                task["status"]
            )
        else:
            if deadline < today:
                deadline_status = "OVERDUE"
            elif deadline == today:
                deadline_status = "DUE TODAY"
            else:
                deadline_status = "Upcoming"
            print(
                i,
                task["title"],
                "-",
                task["deadline"],
```

```
        "-",
        task["time"],
        "-",
        deadline_status,
        "-",
        task["priority"],
        "-",
        task["status"]
    )
```

The view_tasks() function displays all the tasks stored in the system. It first checks if there are no tasks and displays "No tasks available." if the list is empty. If tasks exist, it loops through each task, converts the deadline into a date, and checks whether the task is Completed, Overdue, Due Today, or Upcoming by comparing the deadline with today's date. Finally, it displays the task number, title, deadline, time, deadline status, priority, and current status.

Snippet 3 – Task Summary



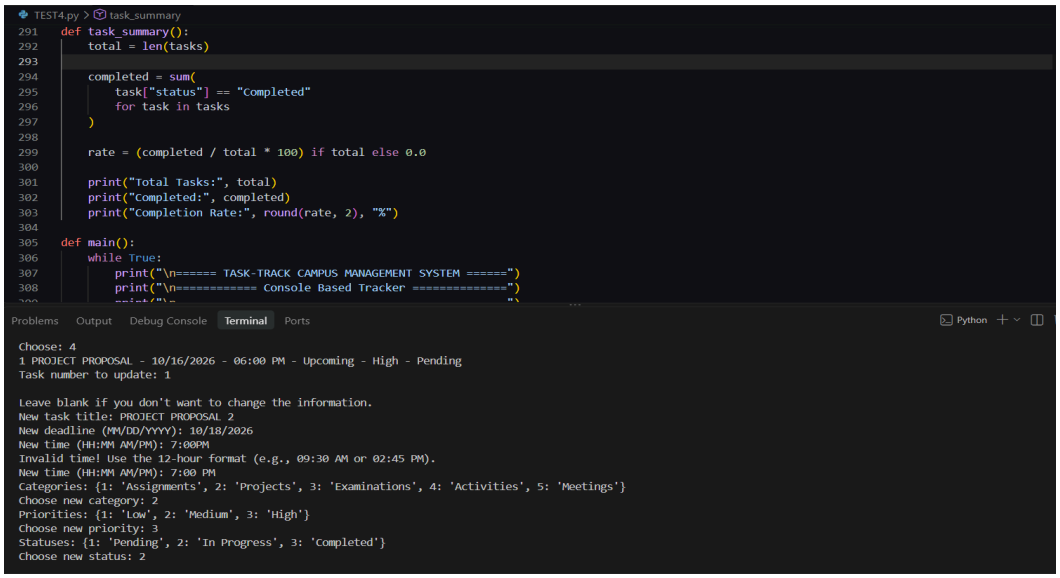
Related Objective: To develop a task summary feature that calculates and displays the total number of tasks, completed tasks, and overall completion rate.

Purpose: The purpose of the Task Summary feature is to help users monitor their task progress and determine the percentage of tasks they have completed.

```
def task_summary():
    total = sum(1 for task in tasks)
    completed = sum(
        task["status"] == "Completed"
        for task in tasks
    )
    rate = (completed / total * 100) if total else 0.0
    print("Total Tasks:", total)
    print("Completed:", completed)
    print("Completion Rate:", round(rate, 2), "%")
```

The task_summary() function provides a summary of the tasks in the system. It first counts the total number of tasks using sum(), then counts how many tasks have a "Completed" status. It calculates the completion rate by dividing the number of completed tasks by the total number of tasks and multiplying it by 100. If there are no tasks, the completion rate is set to 0.0 to avoid an error. Finally, it displays the total number of tasks, completed tasks, and the completion rate rounded to two decimal places.

Snippet 4 - Update Task



Related Objective: The objective is to provide flexibility in managing tasks by allowing users to correct, modify, or update task details whenever necessary, keeping the task information accurate and up to date.

Purpose: The update_task() function allows users to edit and update the information of an existing task, including its title, deadline, time, category, priority, and status.

```
def update_task():
    view_tasks()
    if tasks:
        try:
            num = int(input("Task number to update: "))
            if not 1 <= num <= len(tasks):
                print("Invalid task number.")
                return
            task = tasks[num - 1]
            print(
                "\nLeave blank if you don't want "
                "to change the information."
            )
            title = input("New task title: ")
            if title:
                task["title"] = title
            while True:
                deadline = input("New deadline (MM/DD/YYYY): ")
                if not deadline:
                    break
                try:
                    deadline_date = datetime.strptime(
                        deadline, "%m/%d/%Y"
                    ).date()
```

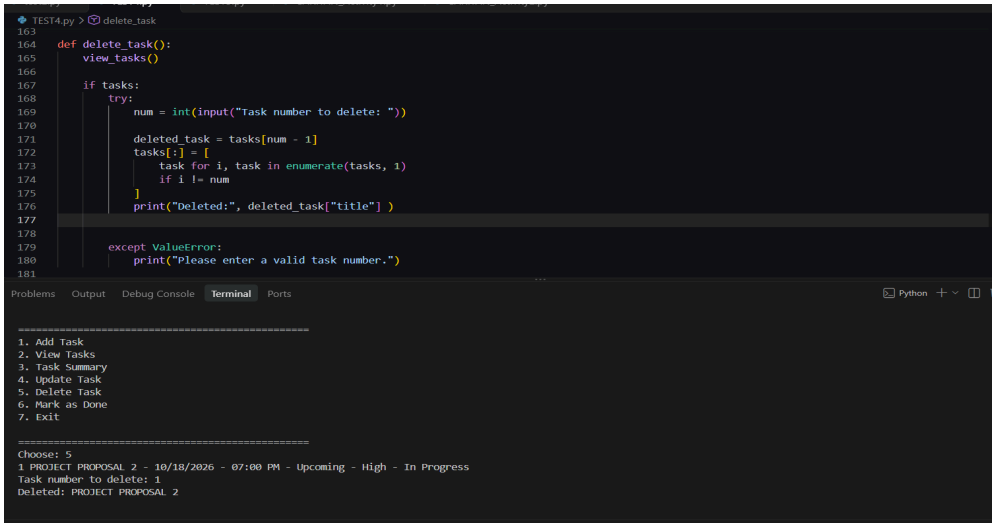
```
            if deadline_date.year not in (2026, 2027):
                print("Invalid year! Only 2026 or 2027 is allowed.")
                continue
            if deadline_date.strftime("%m/%d/%Y") != deadline:
                print("Invalid format! Please use MM/DD/YYYY.")
                continue
            task["deadline"] = deadline
            break
        except ValueError:
            print("Invalid date! Please use MM/DD/YYYY.")
    while True:
        time = input("New time (HH:MM AM/PM): ")
        if not time:
            break
        try:
            valid_time = datetime.strptime(time, "%I:%M %p")
            task["time"] = valid_time.strftime("%I:%M %p")
            break
        except ValueError:
            print("Invalid time! Use the 12-hour format (e.g., 09:30 AM or 02:45 PM).")
    print("Categories:", categories)
    category = input("Choose new category: ")
    if category:
        try:
            category_choice = int(category)
            if category_choice in categories:
                task["category"] = categories[category_choice]
            else:
                print("Invalid category. Keeping the previous value.")
        except ValueError:
            print("Invalid category. Keeping the previous value.")
```

```
        print("Priorities:", priorities)
        priority = input("Choose new priority: ")
        if priority:
            try:
                priority_choice = int(priority)
                if priority_choice in priorities:
                    task["priority"] = priorities[priority_choice]
                else:
                    print("Invalid priority. Keeping the previous value.")
            except ValueError:
                print("Invalid priority. Keeping the previous value.")
        print("Statuses:", statuses)
        status = input("Choose new status: ")
        if status:
            try:
                status_choice = int(status)
                if status_choice in statuses:
                    task["status"] = statuses[status_choice]
                else:
                    print("Invalid status. Keeping the previous value.")
            except ValueError:
                print("Invalid status. Keeping the previous value.")
        print("Task updated!")
    except ValueError:
        print("Please enter a valid task number.")
```

The update_task() function allows the user to modify an existing task. It first displays all available tasks and asks the user to select a task number, then checks if the number is valid. Once a task is selected, the user can enter new information for the title, deadline, time, category, priority, and status. If the user leaves a field blank, the existing information remains unchanged. The selected category, priority, and status are retrieved from their respective dictionaries, and the

updated information is saved to the chosen task. If the task number is invalid, the program displays "Invalid task number." Otherwise, it confirms the update with "Task updated!".

Snippet 5 - Delete Task



Related Objective: The objective is to help users track their progress by updating completed tasks and keeping their task status accurate.

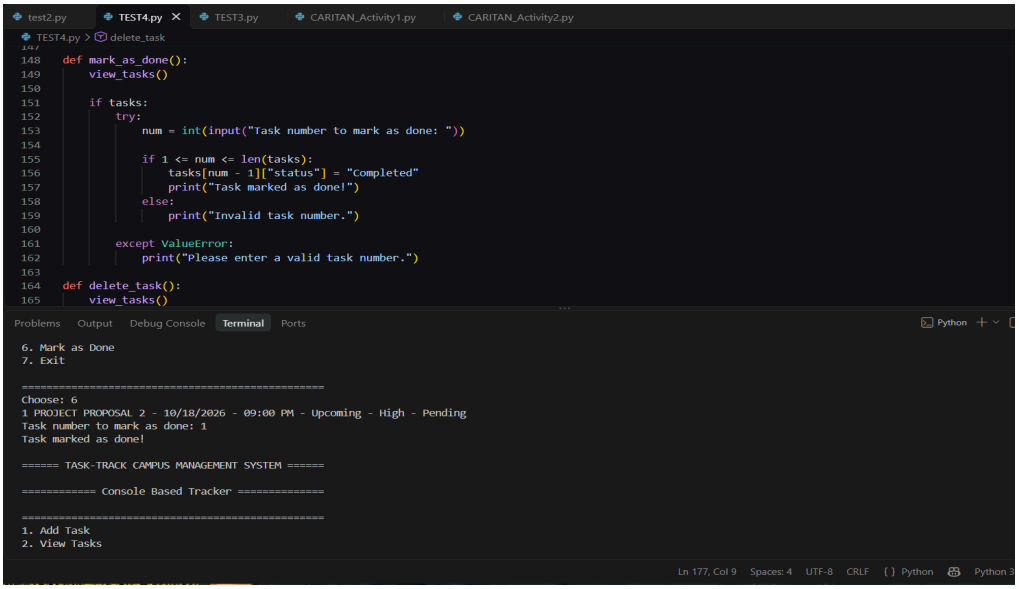
Purpose: The mark_as_done() function allows the user to change the status of a selected task to “Completed.”

```
def delete_task():
    view_tasks()
    if tasks:
        try:
            num = int(input("Task number to delete: "))
            deleted_task = tasks[num - 1]
            tasks[:] = [
                task for i, task in enumerate(tasks, 1)
                if i != num
            ]
```

```
        print("Deleted:", deleted_task["title"])
    except ValueError:
        print("Please enter a valid task number.")
```

The delete_task() function allows the user to remove a task from the task list. It first displays all available tasks and asks the user to enter the number of the task they want to delete. It checks if the entered number is valid, then stores the title of the selected task before creating a new list containing all tasks except the selected one. The original tasks list is then cleared and replaced with the remaining tasks. Finally, it displays the title of the deleted task, while an invalid task number results in an "Invalid task number." message.

Snippet 6 - Mark Task as Done



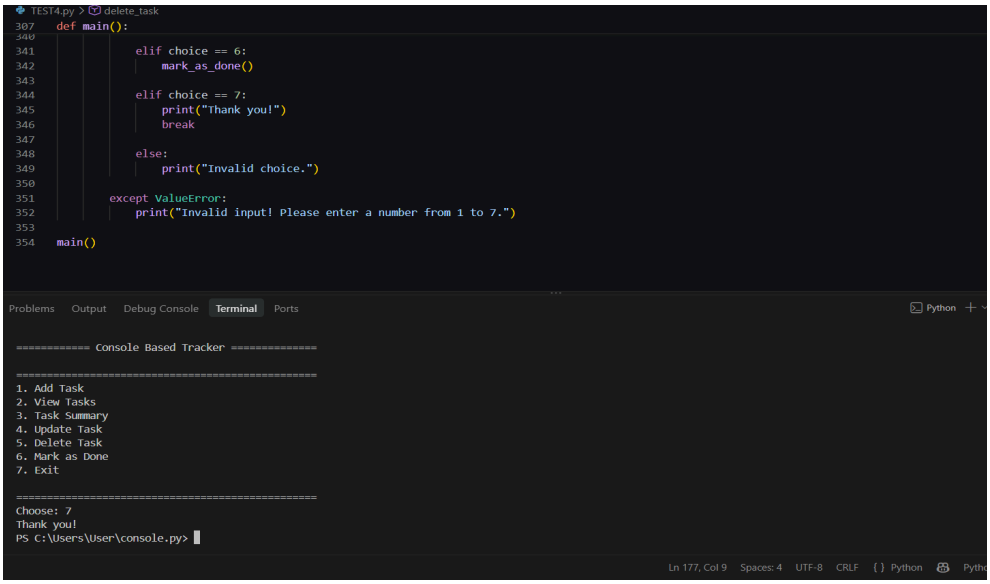
Related Objective: The objective is to track task progress by updating the task status from its current status to “Completed.

Purpose: The mark_as_done() function allows the user to mark a selected task as completed.

```
def mark_as_done():
    view_tasks()
    if tasks:
        num = int(input("Task number to mark as done: "))
        total_tasks = sum(1 for task in tasks)
        if 1 <= num <= total_tasks:
            tasks[num - 1]["status"] = "Completed"
            print("Task marked as done!")
        else:
            print("Invalid task number.")
```

The delete_task() function allows the user to remove a task from the task list. It first displays all available tasks and asks the user to enter the number of the task they want to delete. It checks if the entered number is valid, then stores the title of the selected task before creating a new list containing all tasks except the selected one. The original tasks list is then cleared and replaced with the remaining tasks. Finally, it displays the title of the deleted task, while an invalid task number results in an "Invalid task number." message.

Snippet 7 - Exit



Related Objective: The objective is to provide a simple and clear way for users to safely terminate the program after completing their

Purpose: The elif choice == 7 statement allows the user to exit the Task-Track Campus system by ending the program loop.

```
elif choice == 7:
    print("Thank you!")
    break
```

The elif choice == 7: checks if the user selected option 7 from the menu. If the choice is 7, the program displays "Thank you!" and then uses break to stop the loop, which ends the program or returns control after the menu.

References

- John Paul E. Santos, Joseph A. Villarama, Joseph P. Adsuara, Jordan F. Gundran, Aileen G. De Guzman, Evelyn M. Ben. *Students' Time Management, Academic Procrastination, and Performance during Online Science and Mathematics* https://doi.org/10.26803/ijlter.21.12.8
- Khaldi, M. & Erradi, M. (2020). *International Journal of Emerging Technologies in Learning (iJET), 15*(19), 95-106. https://www.learntechlib.org/p/217929/ Syed Sohail Ahmed Shah, Jannat Malookhani,
- Muniza Memon, & Salman Qazi. (2026). *Design and Implementation of Web Based Task Management System Using Agile Principle*. Spectrum of Engineering Sciences. https://thesesjournal.com/index.php/1/article/view/2075